

```
template<typename D>
class STACK;
```

```
template<typename D>
union LIFO
{
```

```
    struct Continuous { friend class STACK<D>;
```

```
    protected:
```

```
        ::ARRAY<D> S;
```

```
    public:
```

```
        Continuous(size_t size=0) :S(size) {}
```

```
        size_t size();
```

```
        bool empty();
```

```
        bool push(D val);
```

```
        D tos();
```

```
        D pop();
```

```
        void view(bool v) const;
```

```
        void clear();
```

```
        Continuous(const Continuous&) = default;
```

```
        Continuous& operator=(const Continuous&) = default;
```

```
        Continuous(Continuous&&) noexcept = default;
```

```
        Continuous& operator=(Continuous&&) noexcept = default;
```

```
    } array;
```

```
    struct Linked { friend class STACK<D>;
```

```
    protected:
```

```
        ::LIST<D> S;
```

```
    public:
```

```
        Linked()=default;
```

```
        size_t size();
```

```
        bool empty();
```

```
        bool push(D val);
```

```
        D tos();
```

```
        D pop();
```

```
        void view(bool v) const;
```

```
        void clear();
```

```
        Linked(const Linked&) = default;
```

```
        Linked& operator=(const Linked&) = default;
```

```
        Linked(Linked&&) noexcept = default;
```

```
        Linked& operator=(Linked&&) noexcept = default;
```

```
    } list;
```

```
LIFO() {}
```

```
~LIFO() {}
```

```
LIFO(const LIFO&)=delete;
```

```
LIFO& operator=(const LIFO&)=delete;
```

```
LIFO(LIFO&&)=delete;
```

```
LIFO& operator=(LIFO&&)=delete;
```

```
};
```

✓ When you write = default for a copy or move constructor (or assignment operator), you are telling the C++ compiler to automatically create that function. The compiler does not simply copy the raw memory; instead, it copies or moves each member one by one. If a member, such as ARRAY or LIST, has its own custom copy or move constructor, the compiler automatically calls that custom function. So, = default means "generate the function for me," while each member still uses its own copy or move logic if it has one.

- A house (LIFO) contains two rooms (Continuous and Linked).
- You can say, "The whole house cannot be moved."
- That doesn't mean "The furniture inside each room cannot be moved."

✓ LIFO deletes its copy and move constructors and assignment operators to prevent the entire union object from being copied or moved. A union can contain only one active member (array or list) at a time, so disabling these operations avoids accidental misuse. This does not affect the Continuous or Linked classes inside the union, because they are separate types with their own copy and move functions. They can still use their own = default constructors and assignment operators or their custom implementations independently of LIFO.

```

template<typename D>
class STACK{
protected:
    bool C;
    LIFO<D> Stack;
public:
    STACK(size_t size=0);
    ~STACK();

    bool resize(size_t size, bool force=false);
    size_t size();
    bool empty();
    bool push(D val);
    void view(bool v=false) const;
    D tos();
    D pop();
    void clear();

    STACK(const STACK&);
    STACK& operator=(const STACK&);

    STACK(STACK&&) noexcept;
    STACK& operator=(STACK&&) noexcept;
};

```

```

template<typename D>
STACK<D>::STACK(size_t size)
{
    if(size){
        new (&Stack.array) typename LIFO<D>::Continuous(size);
        C=true;
    }else{
        new (&Stack.list) typename LIFO<D>::Linked();
        C=false;
    }
}

```

```

template<typename D>
STACK<D>::~~STACK()
{
    if(C)
        Stack.array.~Continuous();
    else
        Stack.list.~Linked();
}

```

```

template<typename D>
STACK<D>::STACK(const STACK& other)
{
    C=other.C;
    if(C)
        new (&Stack.array) typename LIFO<D>::Continuous(other.Stack.array);
    else
        new (&Stack.list) typename LIFO<D>::Linked(other.Stack.list);
}

```

```

template<typename D>
STACK<D>& STACK<D>::operator=(const STACK& other)
{
    if(this == &other) return *this;

    if(C)
        Stack.array.~Continuous();
    else
        Stack.list.~Linked();

    C=other.C;
    if(C)
        new (&Stack.array) typename LIFO<D>::Continuous(other.Stack.array);
    else
        new (&Stack.list) typename LIFO<D>::Linked(other.Stack.list);

    return *this;
}

```

```

template<typename D>
STACK<D>::STACK(STACK&& other) noexcept
{
    C=other.C;
    if(C)
        new (&Stack.array) typename LIFO<D>::Continuous(std::move(other.Stack.array));
    else
        new (&Stack.list) typename LIFO<D>::Linked(std::move(other.Stack.list));
}

```

```

template<typename D>
STACK<D>& STACK<D>::operator=(STACK&& other) noexcept
{
    if(this == &other) return *this;

    if(C)
        Stack.array.~Continuous();
    else
        Stack.list.~Linked();

    C=other.C;
    if(C)
        new (&Stack.array) typename LIFO<D>::Continuous(std::move(other.Stack.array));
    else
        new (&Stack.list) typename LIFO<D>::Linked(std::move(other.Stack.list));

    return *this;
}

```

- ✓ STACK defines its own copy and move constructors and assignment operators because it contains a LIFO object, whose copy and move operations are deleted. Therefore, the compiler cannot generate these functions automatically for STACK. By writing them manually, STACK can check which storage type (array or list) is currently active and correctly copy or move only that active member, allowing STACK objects to be copied and moved safely.

```

template<typename D>
size_t LIFO<D>::Continuous::size(){ return S.size(); }

template<typename D>
size_t LIFO<D>::Linked::size(){ return S.size(); }

```

```

template<typename D>
size_t STACK<D>::size() {
    return C? Stack.array.size()
           : Stack.list.size();
}

```

```

template<typename D>
union FIFO
{
    struct Continuous { friend class QUEUE<D>;
    protected:
        D *Q;
        size_t C, S, f, r;
    public:
        Continuous(size_t size=2)
            : C(size ? size : 2), S(0), f(0), r(0)
        { Q=new D[C]; }
        ~Continuous() { delete[] Q; }

        size_t size();
        bool empty();
        bool enqueue(D val);
        D dequeue();
        D front();
        D rear();
        void view(bool v) const;
        void clear();

        Continuous(const Continuous&);
        Continuous& operator=(const Continuous&);

        Continuous(Continuous&&) noexcept;
        Continuous& operator=(Continuous&&) noexcept;
    }
    array;

    struct Linked { friend class QUEUE<D>;
    protected:
        ::LIST<D> Q;
    public:
        Linked()=default;

        size_t size();
        bool empty();
        bool enqueue(D val);
        D dequeue();
        D front();
        D rear();
        void view(bool v) const;
        void clear();

        Linked(const Linked&) = default;
        Linked& operator=(const Linked&) = default;

        Linked(Linked&&) noexcept = default;
        Linked& operator=(Linked&&) noexcept = default;
    }
    list;

    FIFO() {}
    ~FIFO() {}

    FIFO(const FIFO&)=delete;
    FIFO& operator=(const FIFO&)=delete;

    FIFO(FIFO&&)=delete;
    FIFO& operator=(FIFO&&)=delete;
};

```

- ✓ In STACK, ARRAY already provides its own copy and move constructors and assignment operators. Therefore, Continuous can simply use = default, which automatically calls ARRAY's custom copy and move functions.
- ✓ In QUEUE, however, the Continuous implementation directly manages dynamic memory using a raw pointer (D* Q). Since raw pointers do not automatically perform deep copying or safe moving, the compiler-generated = default functions would only copy the pointer address, leading to problems such as double deletion. Therefore, QUEUE::Continuous must provide its own copy and move implementations to correctly allocate, copy, transfer, and release the dynamically allocated memory.

```

template<typename D>
FIFO<D>::Continuous::Continuous(const Continuous& other)
    : C(other.C), S(other.S), f(other.f), r(other.r)
{
    Q=new D[C];
    for(size_t i=0; i < C; ++i) Q[i]=other.Q[i];
}

```

```

template<typename D>
typename FIFO<D>::Continuous&
FIFO<D>::Continuous::operator=(const Continuous& other)
{
    if(this != &other){
        D* temp=new D[other.C];
        for(size_t i=0; i < other.C; ++i) temp[i]=other.Q[i];

        delete[] Q;
        Q=temp; C=other.C; S=other.S; f=other.f; r=other.r;
    }
    return *this;
}

```

```

template<typename D>
FIFO<D>::Continuous::Continuous(Continuous&& other) noexcept
    : Q(other.Q), C(other.C), S(other.S), f(other.f), r(other.r)
{
    other.Q=nullptr; other.C=0; other.S=0; other.f=0; other.r=0;
}

```

```

template<typename D>
typename FIFO<D>::Continuous&
FIFO<D>::Continuous::operator=(Continuous&& other) noexcept
{
    if(this != &other){ delete[] Q;
        Q=other.Q; C=other.C; S=other.S; f=other.f; r=other.r;
        other.Q=nullptr; other.C=0; other.S=0; other.f=0; other.r=0;
    }
    return *this;
}

```